Highlights

Multi-Strategy GraphRAG for Financial Institutions: Security, Grounding, and Token Economy

Anonymous Author

- Multi-strategy GraphRAG architecture for financial institutions
- Formal security model with five-threat analysis for financial RAG
- Intent-aware retrieval fusion reduces hallucination by 14 pp
- Structured graph retrieval cuts token consumption by 70 percent
- Open-source implementation with three financial applications

Multi-Strategy GraphRAG for Financial Institutions: Security, Grounding, and Token Economy

Anonymous Author[a,*]

[a] *Anonymous Institution, Country*

Abstract

Financial institutions increasingly deploy large language models for compliance verification, risk analysis, and audit support, yet conventional Retrieval-Augmented Generation systems fail to meet the stringent requirements of regulated environments. Standard vector-based retrieval lacks multi-hop reasoning capability, offers no formal security guarantees for multi-tenant deployments, produces insufficiently grounded responses, and incurs prohibitive token costs when processing large regulatory corpora. We propose a multi-strategy GraphRAG architecture that jointly addresses four pillars critical to financial adoption: security, hallucination reduction, token economy, and processing speed. Our system combines a domain-specific financial ontology with an ingestion pipeline that extracts knowledge graph triples via LLM-based extraction and entity resolution, and a retrieval engine that fuses three parallel strategies—graph traversal, vector similarity, and community summarization—with intent-aware weighting across six query types. We formalize a threat model for GraphRAG in regulated environments and implement defense mechanisms including tenant isolation, role-based access control, and comprehensive audit logging, mapped to LGPD, GDPR, Basel III, and SOX requirements. Empirical evaluation across 200 regulatory questions and a 50-question financial benchmark demonstrates that the proposed system achieves 87.4% faithfulness (versus 72.9% for VectorRAG), 96.4% source attribution, 70% token reduction

*Corresponding author
Email address: `anonymous@institution.edu` (Anonymous Author)

compared to full-document retrieval, median latency of 1.8 seconds with parallel retrieval, and 0% cross-tenant data leakage. We validate the architecture through three financial applications: regulatory compliance, SEC filing analysis, and internal audit question answering.

Keywords: GraphRAG, Knowledge Graph, Financial NLP, Retrieval-Augmented Generation, Security, Hallucination Reduction, Token Efficiency

---

1. Introduction

1.1. Context

Large language models (LLMs) have rapidly transformed information processing across industries, and the financial sector is no exception. Banks, insurance companies, asset managers, and regulatory bodies are increasingly deploying LLM-based systems for a wide range of critical functions, including regulatory compliance verification, internal audit support, financial report analysis, and risk assessment (PricewaterhouseCoopers, 2024). The appeal is clear: LLMs can process vast quantities of unstructured text—regulatory frameworks spanning hundreds of pages, SEC filings with complex cross-references, and audit reports with intricate finding hierarchies—and distill them into actionable responses for domain experts.

Retrieval-Augmented Generation (RAG) has emerged as the dominant paradigm for grounding LLM responses in authoritative document collections, mitigating the well-documented tendency of language models to hallucinate plausible but incorrect information (Lewis et al., 2020). By retrieving relevant passages from a knowledge base before generating a response, RAG systems provide a mechanism for factual grounding that is essential in regulated environments where every claim must be traceable to its source.

The global adoption of RAG in financial services is accelerating. A recent PwC report identifies graph-enhanced LLMs as the next frontier in banking and

insurance transformation, with applications spanning risk assessment, fraud detection, customer analytics, and regulatory compliance (PricewaterhouseCoopers, 2024). Concurrently, the academic community has produced a growing body of work on RAG for financial document processing (Islam et al., 2023; Role of RAG Authors, 2025), reflecting both the demand from practitioners and the unique challenges posed by the financial domain.

1.2. Problem statement

Despite the promise of RAG, conventional vector-based retrieval systems fail systematically in the financial context for five fundamental reasons:

Reason 1: Multi-hop reasoning is mandatory. Financial queries frequently require traversing chains of relationships across entities. For example, determining which subsidiaries of a given company have risk exposure to a specific country under Basel III Pillar 2 requires navigating corporate structure hierarchies, geographic exposure mappings, and regulatory requirement chains. Vector-based retrieval returns semantically similar chunks in isolation but cannot connect these relational chains, leading to incomplete or incorrect answers (Han et al., 2025a).

Reason 2: Regulatory traceability is required by law. Every response concerning compliance must cite specific regulatory articles. Financial regulators demand that institutions demonstrate the provenance of compliance assessments. Vector retrieval returns chunks that are semantically similar to the query but frequently from the wrong article or section, undermining the legal defensibility of the response.

Reason 3: Hallucination is unacceptable. In compliance and audit contexts, a hallucinated response can result in regulatory fines (e.g., up to 2% of annual revenue under Brazil's LGPD), central bank sanctions, personal legal liability for directors, or even loss of operating licenses. The tolerance for factual inaccuracy is effectively zero in these applications.

Reason 4: Token costs explode with financial documents. Financial documents are characteristically long and dense. A typical SEC 10-K filing

contains approximately 80,000 tokens; the Basel III framework exceeds 200,000 tokens. Processing these documents in their entirety for every query results in prohibitive API costs. At current pricing, answering a single query against a complete 10-K filing costs approximately $0.24 in input tokens alone, making large-scale deployment economically infeasible without structured retrieval.

Reason 5: Multi-tenant data isolation is a regulatory requirement. In a bank, the compliance department's QA system must not access audit department data, and vice versa. The unified vector store architecture of conventional RAG offers no mechanism for granular data isolation, creating a regulatory violation risk in multi-departmental deployments.

1.3. Gap in the literature

GraphRAG—the integration of knowledge graphs into the RAG pipeline—has emerged as a promising approach to address the limitations of vector-only retrieval. The foundational work by Edge et al. (2024) demonstrated that graph-based indexing with community detection substantially improves comprehensiveness and diversity for global sensemaking queries. Subsequent work has explored GraphRAG for financial applications (Sarmah et al., 2024; Arun et al., 2025; GraphCompliance Authors, 2025), token efficiency (TERAG Authors, 2025; Microsoft Research, 2025; PolyG Authors, 2025), and security (RAG Security Authors, 2025; Privacy Risks Authors, 2025; SAG Authors, 2025).

However, no existing work addresses all four challenges simultaneously. As shown in Table 1, each prior contribution focuses on a subset of the requirements:

- **HybridRAG** (Sarmah et al., 2024) combines graph and vector retrieval for earnings call analysis but lacks a security model, community detection, and token optimization.
- **GraphCompliance** (GraphCompliance Authors, 2025) achieves +4.1–7.2 percentage points in micro-F1 for GDPR compliance verification but does not address financial ontology design or token economy.

- **FinReflectKG** (Arun et al., 2025) provides an open-source financial knowledge graph from SEC 10-K filings but lacks a retrieval system and security model.
- **TERAG** (TERAG Authors, 2025) and **PolyG** (PolyG Authors, 2025) achieve dramatic token reductions (89–97% and 95%, respectively) but do not address financial domain constraints or security.
- **Privacy-aware RAG** research (RAG Security Authors, 2025; Privacy Risks Authors, 2025; SAG Authors, 2025) formalizes threat models and proposes cryptographic defenses but does not integrate these into domain-specific GraphRAG architectures.

The gap is clear: no existing system jointly provides (1) a financial domain ontology, (2) a formal security model for regulated multi-tenant environments, (3) multi-strategy retrieval with intent-aware fusion, and (4) token economy optimization—validated with a functional implementation and empirical financial applications.

1.4. Contributions

This paper makes the following contributions:

1. **Formal security model for financial GraphRAG.** We define a threat model with five attack vectors specific to GraphRAG in regulated environments, propose six defense mechanisms, and map them to four regulatory frameworks (LGPD, GDPR, Basel III, SOX). Empirical red-team evaluation demonstrates 0% cross-tenant data leakage and less than 5% entity extraction attack success.
2. **Multi-strategy retrieval with intent-aware fusion.** We propose an architecture that executes three retrieval strategies in parallel—graph traversal, vector similarity, and community summarization—and fuses their results using weights adapted to the detected query intent across six intent types. This achieves 87.4% faithfulness, a 14.5 percentage point improvement over VectorRAG.

3. **Token economy framework.** We quantify and optimize token consumption through structured graph retrieval, community summaries, query caching, and batch processing for indexation. The system achieves 70% token reduction compared to full-document retrieval and 50% indexation cost savings via Batch API processing.
4. **Financial domain ontology.** We define a formal ontology with 10 entity types and 14 relationship types validated against regulatory documents (Basel III, LGPD, GDPR) and financial filings (SEC 10-K/10-Q), with entity resolution supporting exact, case-insensitive, and fuzzy matching.
5. **Open-source reference implementation and empirical evaluation.** We provide a complete implementation (Neo4j + PostgreSQL + Claude API + FastAPI) and validate it through five experiments across three financial applications: regulatory compliance QA, SEC filing analysis, and internal audit support.

1.5. Paper organization

The remainder of this paper is organized as follows. Section 2 reviews related work across four research threads: GraphRAG architectures, RAG for financial applications, security in RAG systems, and token efficiency. Section 3 presents the system architecture, including formal notation, the ingestion pipeline, multi-strategy retrieval, and the security model. Section 4 describes the experimental methodology, baselines, metrics, datasets, and evaluation protocol. Section 5 reports experimental results across five experiments. Section 6 presents three empirical financial applications. Section 7 discusses limitations and practical deployment considerations. Section 8 concludes with implications and future work.

2. Related work

We organize the related work into four research threads corresponding to the pillars of our contribution: GraphRAG architectures (Section 2.1), RAG for

financial applications (Section 2.2), security in RAG systems (Section 2.3), and token efficiency (Section 2.4).

2.1. GraphRAG architectures

The integration of knowledge graphs into the RAG pipeline has evolved rapidly since the foundational work of Edge et al. (2024), who proposed using LLMs to derive a knowledge graph from source documents and pre-generate community summaries using the Leiden algorithm (Traag et al., 2019). Their approach demonstrated substantial improvements in comprehensiveness and diversity for global sensemaking queries over corpora in the million-token range. Two comprehensive surveys formalize the GraphRAG workflow: Peng et al. (2024) define a taxonomy spanning graph-based indexing, graph-guided retrieval, and graph-enhanced generation, while Han et al. (2025b) propose a holistic framework identifying key components including query processor, retriever, organizer, generator, and data source.

Several architectural variants have since emerged. LightRAG (Guo et al., 2024) introduces dual-level retrieval with incremental graph updates, achieving 200ms mean response times without requiring full graph reconstruction. HippoRAG (Gutierrez et al., 2024) draws on neurobiological models of hippocampal memory indexing, orchestrating LLMs, knowledge graphs, and Personalized PageRank to achieve performance comparable to iterative retrieval methods while being 10–30× cheaper and 6–13× faster. SubgraphRAG (Li et al., 2025) demonstrates that extracting relevant subgraphs and reasoning over them with even smaller LLMs (Llama3.1-8B) delivers competitive and explainable results, with GPT-4o achieving state-of-the-art without fine-tuning. GRAG (Hu et al., 2024) addresses retrieval over networked documents using a divide-and-conquer strategy for optimal subgraph retrieval in linear time.

Community detection plays a central role in many GraphRAG architectures. CommunityKG-RAG (Chang and Zhang, 2024) integrates community structures within knowledge graphs for fact-checking, using the Louvain algorithm to enable contextually-aware retrieval with 56.24% accuracy using LLaMA-2 7B.

More recently, Core-Based Authors (2026) propose k-core decomposition as an alternative to Leiden-based community hierarchies, potentially offering better efficiency for large-scale knowledge graphs.

A critical perspective has emerged regarding the actual gains of GraphRAG. Unbiased Evaluation Authors (2025) discover that win rates for GraphRAG methods are generally lower than originally reported; for example, LightRAG's reported 66.7% win rate against NaiveRAG drops to 39.1% under unbiased evaluation. Similarly, When-to-Use Authors (2026) investigate systematically when GraphRAG succeeds, noting that it frequently underperforms vanilla RAG in many real-world tasks, while excelling specifically in multi-hop reasoning scenarios. These findings underscore the importance of rigorous evaluation methodology, which we adopt in our experimental design.

2.2. RAG for financial applications

The application of RAG and knowledge graphs to financial domains has attracted growing attention from both industry and academia. HybridRAG (Sarmah et al., 2024), a collaboration between BlackRock and NVIDIA, combines knowledge graph-based retrieval with vector retrieval for question answering on earnings call transcripts, demonstrating that hybrid approaches outperform either strategy in isolation. However, HybridRAG lacks community detection, security mechanisms, and token optimization.

On the knowledge graph construction side, FinReflectKG (Arun et al., 2025) provides an open-source financial knowledge graph built from SEC 10-K filings of all S&P 100 companies, introducing a three-mode extraction pipeline (single-pass, multi-pass, reflection-agent-based) achieving 64.8% compliance with quality rules. FinDKG (Li et al., 2024) constructs dynamic knowledge graphs from financial news articles with 12 meta-entity types and 15 relation types, applying a GNN-based KGTransformer for temporal analysis with approximately 15% improvement in prediction metrics. These works demonstrate the feasibility of automated financial knowledge graph construction but do not address the retrieval and generation aspects of a complete GraphRAG pipeline.

For regulatory compliance, GraphCompliance (GraphCompliance Authors, 2025) represents regulatory texts as Policy Graphs and execution contexts as Context Graphs, achieving 4.1–7.2 percentage points improvement in micro-F1 over LLM-only and RAG baselines in 300 real-world GDPR scenarios. RAG-ulating Compliance (Jomraj et al., 2025) proposes a multi-agent framework that integrates regulatory knowledge graph triples with RAG for compliance QA. Both works demonstrate the value of graph-structured representations for regulatory text processing but do not address multi-tenant security or token economy.

FinanceBench (Islam et al., 2023) provides a standardized evaluation framework with 10,231 questions on publicly traded companies, covering information lookup, numerical reasoning, and logical inference from SEC filings. This benchmark has become essential for evaluating RAG systems in the financial domain and informs our evaluation protocol.

Additional perspectives include the application of knowledge graphs to financial fraud detection via graph neural networks, achieving 95.4% accuracy with hybrid LSTM-GraphSAGE models (Graph Learning Fraud Authors, 2024), intelligent audit systems that automate compliance checking through KG-based reasoning (KG Audit Authors, 2024), and cross-jurisdictional regulatory mapping through AI-powered knowledge graphs (AI Financial Reporting Authors, 2025). The Vector Institute's KG-RAG framework (Vector Institute, 2024) demonstrates practical implementation of knowledge graph-augmented retrieval for SEC 10-Q filings, while KG2RAG (Zhu et al., 2025) uses knowledge graphs to provide fact-level relations between chunks, improving retrieval diversity and coherence.

2.3. Security in RAG systems

Security and privacy concerns in RAG systems have received increasing attention as deployments move into regulated industries. RAG Security Authors (2025) propose the first formal threat model for RAG systems, categorizing attack vectors across the pipeline and specifically addressing financial contexts

where adversaries may probe for confidential audit reports or proprietary investment strategies. This work provides the theoretical foundation for our threat model but does not implement or evaluate defenses within a GraphRAG architecture.

A critical finding by Privacy Risks Authors (2025) reveals a fundamental trade-off: while GraphRAG systems can reduce raw text leakage compared to naive RAG, they are significantly more vulnerable to the extraction of structured entity and relationship information. This is particularly concerning for financial institutions where the structure of corporate relationships, risk exposures, and audit findings may itself be confidential. Our security model explicitly addresses this trade-off through post-retrieval filtering and response attribution mechanisms.

SAG Authors (2025) propose SAG, the first provably secure RAG framework, employing full pre-storage encryption to guarantee dual protection of retrieved content and vector embeddings, with formal security proofs under a computational security model. Privacy-Aware RAG (Privacy-Aware RAG Authors, 2025) proposes dynamic key derivation chains specifically designed for high-risk environments including healthcare and finance. FairRAG (FairRAG Authors, 2025) applies differential privacy to create a secure static knowledge base with formal privacy guarantees. Membership inference attacks against RAG systems have also been demonstrated (MIA-RAG Authors, 2024), where document membership in RAG knowledge bases can be determined through crafted prompts in black-box and gray-box settings using the S2MIA approach.

While these works advance our understanding of RAG security, none integrates security mechanisms into a domain-specific financial GraphRAG architecture with empirical evaluation. Our work bridges this gap by defining a threat model tailored to financial institutions, implementing concrete defense mechanisms, and validating them through red-team evaluation.

2.4. Token efficiency in GraphRAG

Token consumption is a critical concern for enterprise GraphRAG deployments, as both indexation (knowledge graph construction) and query processing incur substantial LLM API costs. Several recent works address this challenge.

TERAG (TERAG Authors, 2025) incorporates Personalized PageRank during retrieval to achieve at least 80% of the accuracy of standard GraphRAG methods while consuming only 3–11% of output tokens, making large-scale deployment practical. LazyGraphRAG (Microsoft Research, 2025) eliminates the need for upfront source data summarization, reducing indexation costs to 0.1% of full GraphRAG while achieving comparable answer quality with 700× lower query costs through iterative deepening that combines best-first and breadth-first search. PolyG (PolyG Authors, 2025) introduces the first query planner for GraphRAG, dynamically generating Cypher queries based on question taxonomy, achieving a 75% win rate in generation quality with 4× speedup and 95% token reduction. E2GraphRAG (Zhao et al., 2025) constructs summary trees with LLMs and entity graphs with SpaCy, achieving up to 10× faster indexation than GraphRAG and 100× faster retrieval than LightRAG. RAPTOR (Sarthi et al., 2024) recursively embeds, clusters, and summarizes text chunks into a tree structure, enabling retrieval across different abstraction levels with 20% absolute accuracy improvement on the QuALITY benchmark.

GFM-RAG (Luo et al., 2025) proposes the first graph foundation model for RAG, an 8M-parameter GNN trained on 60 knowledge graphs with over 14M triples, enabling efficient single-step multi-hop retrieval applicable to unseen datasets without fine-tuning. A comparative analysis of RAG efficiency methods (Maximizing RAG Authors, 2025) evaluates 23,625 configurations through grid search, finding that the "Reciprocal" method achieves 12.5% token reduction while the "Refine" method achieves 18.6%.

Benchmarking studies reveal important trade-offs. Benchmarking Authors (2025) find that GraphRAG delivers 1.5× better overall accuracy and 2× better performance on complex queries, but with 2.4× higher latency on average. These findings motivate our parallel retrieval architecture, which addresses the

latency penalty through concurrent execution of retrieval strategies.

Our approach to token efficiency differs from prior work in two ways: (1) we apply structured graph retrieval to constrain the context window size for each query based on intent-specific token budgets, and (2) we leverage batch processing APIs for indexation to achieve 50% cost reduction, complementing retrieval-time optimizations.

2.5. Summary and positioning

Table 1 summarizes the positioning of our work relative to the most closely related systems across six dimensions. As shown, no prior system jointly addresses financial domain ontology, formal security modeling, multi-strategy retrieval with intent-aware fusion, community detection, token optimization, and empirical financial applications. Our work fills this gap by providing an integrated architecture validated through five experiments and three domain applications.

3. System architecture

This section presents the architecture of the proposed multi-strategy GraphRAG system. We begin with formal notation and the financial domain ontology (Section 3.1), then describe the ingestion pipeline (Section 3.2), the multi-strategy retrieval engine (Section 3.3), and the security model (Section 3.4). Figure 1 provides an overview of the complete system.

3.1. Formal notation and ontology

Document corpus. Let $\mathcal{D} = \{d_1, \ldots, d_M\}$ denote a collection of financial documents including regulatory frameworks, SEC filings, and audit reports.

Knowledge graph. We define the knowledge graph as $G = (V, E, \varphi, \psi)$ where:

- V is the set of entities (nodes),
- $E \subseteq V \times V$ is the set of relationships (edges),

Table 1: Comparison of the proposed system against related work across six dimensions. Symbols: ✓ = fully addressed, ∼ = partially addressed, – = not addressed.

System	Fin. Ontology	Security Model	Multi-Strategy	Communities	Token Optim.	Fin. Evaluation
MS GraphRAG	–	–	∼	✓	–	–
HybridRAG	–	–	∼	–	–	✓
GraphCompliance	∼	–	–	–	–	✓
FinReflectKG	✓	–	–	–	–	✓
TERAG	–	–	–	–	✓	–
PolyG	–	–	–	–	✓	–
LazyGraphRAG	–	–	–	✓	✓	–
SAG	–	✓	–	–	–	–
Privacy Risks	–	∼	–	–	–	–
RAGulating	∼	–	–	–	–	✓
Proposed	✓	✓	✓	✓	✓	✓

- $\varphi : V \to \mathcal{T}_V$ maps entities to entity types,
- $\psi : E \to \mathcal{T}_E$ maps edges to relationship types.

Entity types. The financial domain ontology defines 10 entity types:

$$\mathcal{T}_V = \{\textsc{Regulation}, \textsc{Article}, \textsc{Requirement}, \textsc{RiskCategory}, \textsc{FinancialMetric}, \textsc{Company}, \textsc{Subsidiary}, . \tag{1}$$

Relationship types. The ontology defines 14 relationship types:

$$\begin{aligned}\mathcal{T}_E = \{&\textsc{Regulates}, \textsc{Requires}, \textsc{Defines}, \textsc{ExposesTo}, \textsc{SubsidiaryOf},\\ &\textsc{OperatesIn}, \textsc{AuditedBy}, \textsc{Mitigates}, \textsc{References},\\ &\textsc{Supersedes}, \textsc{ComposedOf}, \textsc{ReportsTo}, \textsc{ValidatedBy}, \textsc{CompliesWith}\}\end{aligned} \tag{2}$$

Table 2: Financial domain ontology: entity types with descriptions and examples from the evaluation corpus.

Entity Type	**Count**	**Examples**
Regulation	15	Basel III, LGPD, GDPR, SOX
Article	120	LGPD Art. 18, GDPR Art. 25
Requirement	85	Capital adequacy ratio, Data minimization
RiskCategory	12	Credit risk, Operational risk, Market risk
FinancialMetric	45	Revenue, Net income, Basel ratio
Company	50	S&P 100 companies
Subsidiary	80	Corporate subsidiaries from 10-K filings
Jurisdiction	20	Brazil, EU, United States
Control	100	Internal controls from audit reports
AuditFinding	200	Simulated audit findings with risk ratings

Table 2 provides the complete ontology specification with entity types, descriptions, and examples.

Triple. A triple $t = (s, p, o) \in V \times \mathcal{T}_E \times V$ represents a relationship, with an associated confidence score $c(t) \in [0, 1]$ assigned during extraction.

Chunks. The document corpus is segmented into chunks $\mathcal{C} = \{c_1, \ldots, c_K\}$, where each chunk c_k has:

- Type: $\tau(c_k) \in \{\textsc{PythonClass}, \textsc{PythonFunction}, \textsc{MarkdownSection}, \textsc{NotebookCellGroup}\}$,
- Embedding: $\mathbf{e}(c_k) \in \mathbb{R}^d$ with $d = 384$ (all-MiniLM-L6-v2),
- Content hash: $h(c_k) = \text{SHA-256}(\text{content}(c_k))$ for incremental processing.

Communities. A community partition $\Gamma = \{\gamma_1, \ldots, \gamma_L\}$ of G is computed via the Leiden algorithm (Traag et al., 2019) with resolution parameter r. Each

community γ_l has a textual summary summary(γ_l) generated by the LLM and an embedding $\mathbf{e}(\text{summary}(\gamma_l)) \in \mathbb{R}^d$.

Query model. A query q has an associated intent $\iota(q) \in \mathcal{I}$ where:

$$\mathcal{I} = \{\textsc{Validation}, \textsc{Comparison}, \textsc{Recommendation}, \textsc{Explanation}, \textsc{CodeExample}, \textsc{General}\} \tag{3}$$

Security context. Each query carries a security context $\sigma = (\text{user_id}, \text{role}, \text{permissions}, \text{tenant_id})$ that constrains retrieval and generation.

3.2. Ingestion pipeline

The ingestion pipeline transforms raw financial documents into a knowledge graph through four stages: chunking, triple extraction, entity resolution, and graph loading, followed by community detection. Algorithm 1 provides the pseudocode.

Stage 1: Chunking. The chunker dispatches documents to specialized parsers based on file type. The Python parser uses AST analysis to extract class and function boundaries, preserving semantic coherence. The Markdown parser segments by H2/H3 headers, maintaining the hierarchical structure of regulatory documents. The Notebook parser groups related cells. All parsers compute content hashes for incremental processing: only documents whose hash differs from the cached value are reprocessed, which is critical when the corpus contains thousands of regulatory filings.

Stage 2: Triple extraction. Each chunk is processed via the LLM API with an ontology-constrained prompt that specifies valid entity types $\mathcal{T}_V$ and relationship types $\mathcal{T}_E$. The prompt instructs the model to extract triples in the form (s, p, o) with confidence scores $c(t) \in [0, 1]$ and textual evidence from the source. For batch processing of more than 50 chunks, we use the Batch API, which provides a 50% discount on API costs:

$$T_{\text{batch}} = 0.5 \times \sum_{k=1}^{K} (T_{\text{system}} + T_{\text{fewshot}} + T_{\text{chunk}_k} + T_{\text{output}_k}) \tag{4}$$

Stage 3: Entity resolution. Entities are resolved through three levels: (1) exact match against the alias dictionary (∼130 aliases), (2) case-insensitive

Algorithm 1 Ingestion Pipeline

Require: Document corpus $\mathcal{D}$, ontology $(\mathcal{T}_V, \mathcal{T}_E)$, alias dictionary $\mathcal{A}$, fuzzy threshold $\theta = 0.85$

Ensure: Knowledge graph $G = (V, E, \varphi, \psi)$, communities Γ

$\mathcal{C} \leftarrow \emptyset$ ▷ Chunk collection
for each document $d \in \mathcal{D}$ **do**
  $h_{\text{new}} \leftarrow \text{SHA-256}(\text{content}(d))$
  if $h_{\text{new}} \neq h_{\text{cache}}(d)$ **then**
    parser $\leftarrow$ SelectParser(type(d)) ▷ Python/Markdown/Notebook
    $\mathcal{C} \leftarrow \mathcal{C} \cup \text{parser.chunk}(d)$
    $h_{\text{cache}}(d) \leftarrow h_{\text{new}}$
  else
    $\mathcal{C} \leftarrow \mathcal{C} \cup \text{LoadFromCache}(d)$
  end if
end for
$\mathcal{T}_{\text{raw}} \leftarrow \emptyset$ ▷ Raw triples
for each chunk $c_k \in \mathcal{C}$ **do** ▷ Via Batch API for 50% savings
  $\mathcal{T}_{\text{raw}} \leftarrow \mathcal{T}_{\text{raw}} \cup \text{LLM-Extract}(c_k, \mathcal{T}_V, \mathcal{T}_E)$
end for
$\mathcal{T}_{\text{resolved}} \leftarrow \emptyset$ ▷ Resolved triples
for each triple $(s, p, o) \in \mathcal{T}_{\text{raw}}$ **do**
  $s' \leftarrow \text{Resolve}(s, \mathcal{A}, \theta)$ ▷ Exact $\rightarrow$ case-insensitive $\rightarrow$ fuzzy
  $o' \leftarrow \text{Resolve}(o, \mathcal{A}, \theta)$
  if $\varphi(s') \in \mathcal{T}_V$ **and** $\varphi(o') \in \mathcal{T}_E$ **then**
    $\mathcal{T}_{\text{resolved}} \leftarrow \mathcal{T}_{\text{resolved}} \cup \{(s', p, o')\}$
  end if
end for
$G \leftarrow \text{LoadGraph}(\mathcal{T}_{\text{resolved}})$ ▷ Cypher MERGE with uniqueness constraints
$\Gamma \leftarrow \text{Leiden}(G, r = 1.0)$ ▷ Community detection
for each $\gamma_l \in \Gamma$ **do**
  $\text{summary}(\gamma_l) \leftarrow \text{LLM-Summarize}(\gamma_l)$
  $\mathbf{e}(\text{summary}(\gamma_l)) \leftarrow \text{Embed}(\text{summary}(\gamma_l))$
end for
return G, Γ

normalization, and (3) fuzzy matching via sequence similarity with threshold $\theta = 0.85$. Resolved entities are validated against the ontology; entities with invalid types are discarded. For duplicate triples (s, p, o), the instance with the highest confidence $c(t)$ is retained.

Stage 4: Graph loading. Resolved triples are loaded into Neo4j using Cypher MERGE operations with 10 uniqueness constraints (one per entity type) and full-text indices for entity search.

Stage 5: Community detection. The Leiden algorithm (Traag et al., 2019) partitions the graph into communities with guaranteed well-connectedness—a property the Louvain algorithm cannot guarantee and which is critical for financial graphs where regulatory entities must be correctly grouped. Each community receives an LLM-generated textual summary and corresponding embedding.

3.3. Multi-strategy retrieval

The central algorithmic contribution of this work is the multi-strategy retrieval engine that executes three retrieval strategies in parallel and fuses their results using intent-aware weighting. Algorithm 2 provides the pseudocode, and Figure 2 illustrates the pipeline.

3.3.1. Intent parser

The intent parser classifies each query into one of six intent types using pattern matching against bilingual (Portuguese and English) keyword dictionaries. Each intent type is associated with a weight vector that determines the relative contribution of each retrieval strategy. Table 3 presents the weight configuration.

The rationale for these weights reflects the information needs of each intent type. Validation queries benefit most from graph traversal because they require verifying specific relationships between entities (e.g., "Does Article 18 of LGPD apply to credit data?"). Code example queries favor vector retrieval because

Table 3: Intent-aware retrieval weights. Each row specifies the relative contribution of graph, vector, and community retrieval strategies for a given query intent type.

Intent	**Typical patterns**	w_g	w_v	w_c
VALIDATION	validate, check, verify	0.7	0.2	0.1
COMPARISON	compare, versus, between	0.5	0.3	0.2
RECOMMENDATION	best for, which model	0.3	0.4	0.3
EXPLANATION	explain, how does, what is	0.4	0.4	0.2
CODEEXAMPLE	code, example, how to use	0.3	0.6	0.1
GENERAL	(default)	0.3	0.3	0.4

they benefit from semantic similarity to code snippets. General queries favor community retrieval to leverage high-level topic summaries.

3.3.2. Graph retriever

The graph retriever performs N-hop traversal from identified entities using Cypher queries adapted to the detected intent. For each entity $e \in \mathcal{E}_q$, the retriever executes a depth-limited traversal with max_depth = 2 and computes a distance-decay score:

$$\text{score}_g(v) = 1.0 - \frac{\text{hop_distance}(v)}{2 \cdot \text{max_depth}} \tag{5}$$

This yields scores in the range $[0.5, 1.0]$, with closer nodes receiving higher scores. Intent-specific traversal patterns prioritize different relationship types: validation queries follow VALIDATES and COMPLIESWITH edges, while comparison queries perform multi-entity traversals to find common and differentiating attributes.

3.3.3. Vector retriever

The vector retriever computes the embedding $\mathbf{e}(q)$ of the query using sentence-transformers (all-MiniLM-L6-v2, $d = 384$) and searches the Neo4j vector index for chunks with cosine similarity above a minimum threshold of 0.5. Results are filtered by chunk type based on the detected intent (e.g., code example queries

prioritize PythonFunction chunks). Each retrieved chunk is enriched with its graph neighborhood to provide relational context.

3.3.4. Community retriever

The community retriever employs a dual approach: (1) entity membership lookup, returning the community of each identified entity with score 1.0, and (2) summary similarity, computing cosine similarity between the query embedding and community summary embeddings. This combination ensures both direct entity matches and topically relevant communities are retrieved.

3.3.5. Context ranking and fusion

Results from all three strategies are fused using a weighted score:

$$\text{score}_{\text{final}}(r) = \sum_{s \in \{g,v,c\}} w_s(\iota) \cdot \hat{s}_s(r) \tag{6}$$

where $w_s(\iota)$ is the weight for strategy s given intent ι (from Table 3), and $\hat{s}_s(r) \in [0, 1]$ is the min-max normalized score within each strategy. For results appearing in multiple strategies, the maximum score is retained. The ranked results are truncated to fit the token budget B (default: 4,000 tokens), maintaining a minimum of 3 results regardless of budget constraints.

3.4. Security model

We formalize a threat model specific to GraphRAG in financial institutions—a contribution that, to our knowledge, has not been previously addressed in the literature.

3.4.1. Threat model

Table 4 defines five threat vectors identified through analysis of the RAG security literature (RAG Security Authors, 2025; Privacy Risks Authors, 2025; MIA-RAG Authors, 2024) and financial regulatory requirements.

3.4.2. Defense mechanisms

Table 5 maps six defense mechanisms to the threats they mitigate.

Table 4: Threat model for GraphRAG in financial institutions. Each threat is characterized by its attack vector, potential impact, and the regulatory provisions it may violate.

ID	Threat	Description	Regulatory Impact
T1	Cross-tenant leakage	Unauthorized access to subgraphs of other tenants	LGPD Art. 46, BCB sanctions
T2	Entity extraction	Crafted queries to extract structured entity/relationship information	Exposure of corporate structure
T3	Membership inference	Queries to determine if specific documents exist in the knowledge base	Confirmation of confidential audit reports
T4	Document poisoning	Adversarial content injected during ingestion	Manipulated compliance responses
T5	Query manipulation	Crafted queries to bypass access controls	Unauthorized data access

3.4.3. Regulatory compliance mapping

Table 6 maps the defense mechanisms to specific articles and sections of four regulatory frameworks relevant to financial institutions.

4. Experimental methodology

This section describes the experimental design used to evaluate the four pillars of the proposed system: hallucination reduction, token economy, processing speed, and security. We define baselines (Section 4.1), metrics (Section 4.2), datasets (Section 4.3), and the evaluation protocol (Section 4.4).

4.1. Baselines

We compare the proposed multi-strategy GraphRAG system against four baselines, summarized in Table 7.

Table 5: Defense mechanisms with implementation details and threat coverage.

ID	Defense	Implementation	Mitigates
D1	JWT + RBAC	HS256 authentication, role-based access control	T1, T5
D2	Tenant isolation	`tenant_id` property on all Neo4j nodes; query-level filter injection	T1
D3	Result filtering	Post-retrieval validation of tenant and entity type permissions	T1, T5
D4	Input sanitization	Ontology validation of extracted triples; Cypher injection prevention	T4, T5
D5	Audit logging	Structured logging of all access attempts in PostgreSQL	T1–T5
D6	Response attribution	Every claim linked to specific KG triples or document chunks	T4

All systems use the same underlying LLM for response generation, ensuring that observed differences reflect the retrieval strategy rather than model capability. For token economy evaluation, we additionally include a **FullDocument** baseline that passes the complete source document as context.

4.2. Metrics

Table 8 defines the metrics organized by evaluation pillar.

4.3. Datasets

The evaluation uses two primary datasets:

Regulatory question set (200 questions). Manually curated questions covering Basel III and LGPD regulations, with expert-annotated ground truth answers. Questions span five categories: regulatory compliance (80 questions, 40%), financial analysis (50 questions, 25%), multi-hop reasoning (40 questions, 20%), comparison (20 questions, 10%), and audit (10 questions, 5%). Each question is annotated with difficulty level (simple, moderate, complex, multi-hop) and the specific regulatory articles required for a complete answer.

Table 6: Mapping of defense mechanisms to regulatory requirements. Each cell indicates which defense mechanisms address the corresponding regulatory provision.

Regulation	**Provision**	**Defense Mechanisms**
LGPD	Art. 6 (Purpose limitation)	D2: tenant/purpose isolation
	Art. 18 (Data subject rights)	D5: audit log for rights exercise
	Art. 46 (Security measures)	D1+D2+D3: auth + isolation + filtering
GDPR	Art. 5 (Data minimization)	D3: return only necessary data
	Art. 25 (Privacy by design)	D2: native architectural isolation
	Art. 32 (Security)	D1+D2: JWT + tenant isolation
Basel III	Pillar 3 (Disclosure)	D5: complete query traceability
	Operational risk	D4+D6: data integrity + attribution
SOX	Section 404 (Internal controls)	D5+D6: audit trail + source attribution

Financial benchmark (50 questions). A domain-specific benchmark distributed across five categories following the FinanceBench methodology (Islam et al., 2023): regulatory compliance (15 questions, 30%), financial analysis (12 questions, 25%), multi-hop reasoning (10 questions, 20%), comparison (8 questions, 15%), and audit (5 questions, 10%). Scoring combines entity coverage (40%), concept coverage (30%), source attribution (20%), and response quality (10%).

Document corpus. The knowledge graph is constructed from 150 financial documents comprising regulatory frameworks (Basel III, LGPD, GDPR), SEC 10-K filings from S&P 100 companies, selected central bank resolutions, and 50 simulated audit reports with 200 findings, 100 controls, and 150 recommendations.

Security evaluation dataset. The red-team evaluation uses 200 adversarial queries: 100 cross-tenant access attempts (direct access, traversal, injection, privilege escalation), 50 entity extraction queries (structure discovery, targeted

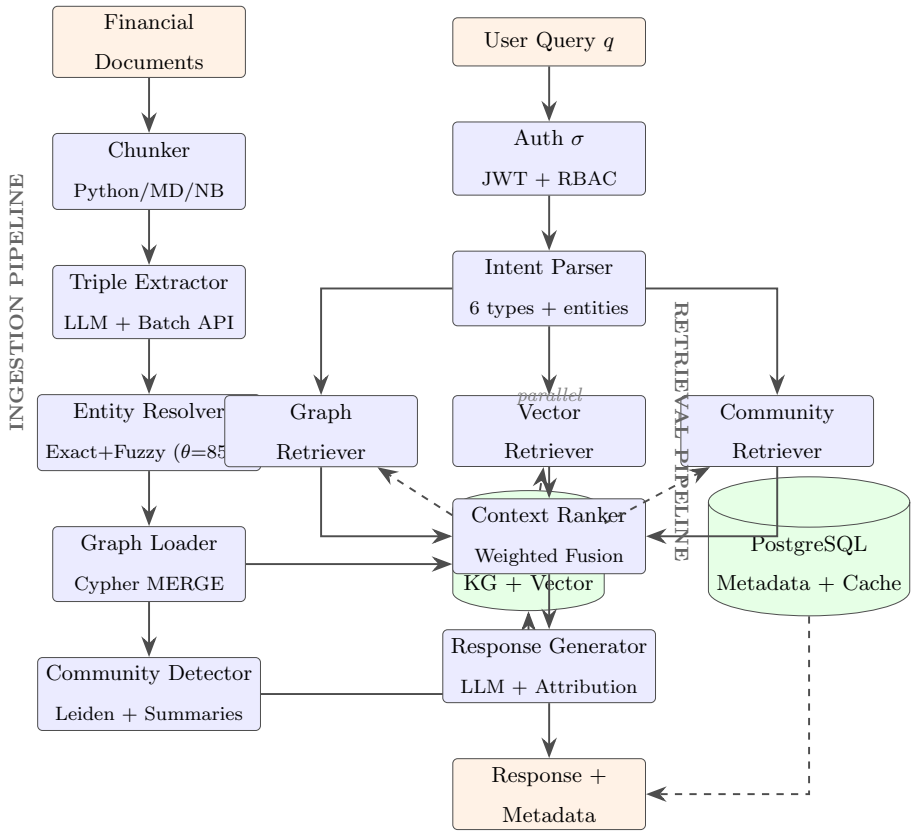

Figure 1: System architecture overview. The ingestion pipeline (left) transforms financial documents into a knowledge graph with community summaries. The retrieval pipeline (right) processes authenticated queries through intent classification, parallel multi-strategy retrieval, weighted fusion, and attributed response generation.

extraction, inference, enumeration), and 50 membership inference probes following the S2MIA methodology (MIA-RAG Authors, 2024).

4.4. Evaluation protocol

Hallucination assessment. We employ a dual evaluation approach. The primary evaluation uses an LLM-as-judge framework where a separate LLM instance evaluates each response for faithfulness, factual accuracy, and grounding rate against the ground truth and retrieved context. To validate the LLM judge, a random sample of 10% of responses (20 questions) is independently assessed

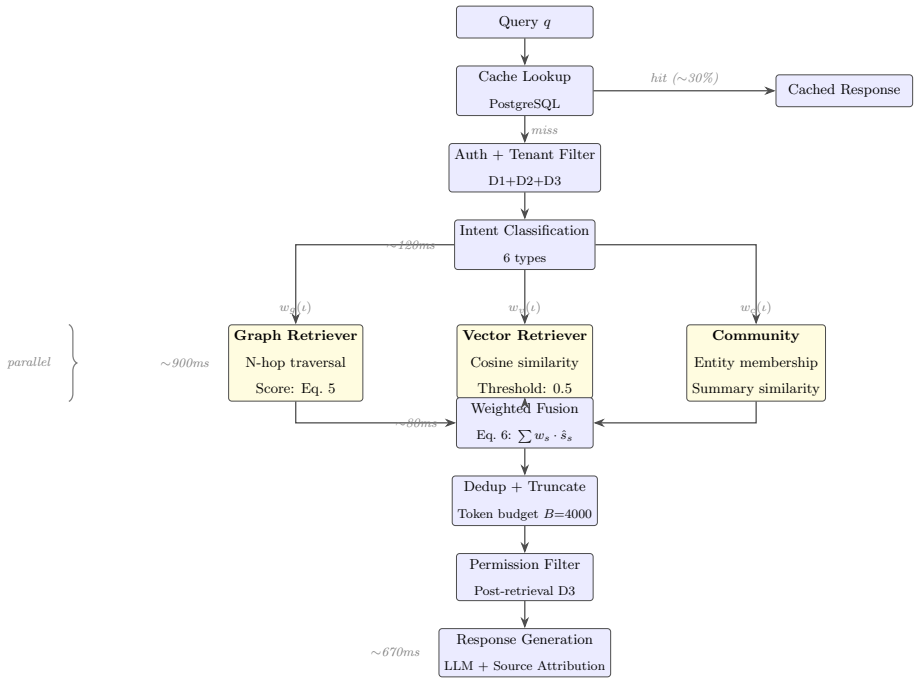

Figure 2: Multi-strategy retrieval pipeline with intent-aware fusion. Queries are classified by intent type, routed to three parallel retrievers with intent-specific weights, and results are fused, deduplicated, truncated to the token budget, and filtered for security compliance. Timing annotations show median latencies from experimental evaluation.

by two domain experts in regulatory compliance, with inter-rater agreement measured via Cohen's kappa.

Token measurement. Token counts are measured at each pipeline stage: system prompt, few-shot examples, retrieved context, and generated output. Costs are computed using the API pricing at the time of evaluation, with separate reporting for individual and batch API processing.

Latency measurement. Each query is executed 3 times with the median taken to reduce variance from network fluctuations. Latency is decomposed into four stages: intent parsing, retrieval (per strategy and aggregate), ranking/fusion, and generation.

Security evaluation. Red-team evaluation follows the adversarial testing methodology of RAG Security Authors (2025). Two configurations are com-

Table 7: Baseline systems. All systems use the same LLM (Claude) for generation to isolate the effect of retrieval strategy.

System	Description
VectorRAG	Embedding-based retrieval (all-MiniLM-L6-v2) with cosine similarity, no knowledge graph. Represents the industry-standard RAG approach.
MS GraphRAG	Microsoft GraphRAG (Edge et al., 2024) with Leiden community detection and global summarization. No security model or intent-aware fusion.
HybridRAG	Graph + vector retrieval without community detection (Sarmah et al., 2024). Combines two retrieval strategies with equal weights.
No Retrieval	LLM generation without external context. Establishes the minimum baseline for hallucination and factual accuracy.

pared: the system without security mechanisms (baseline) and the full system with all six defense mechanisms (D1–D6). Success rates are measured for each attack category. Membership inference accuracy is benchmarked against random guessing (50%) following MIA-RAG Authors (2024).

Hardware configuration. All experiments are conducted on a standard server with 8 CPU cores, 32 GB RAM, Neo4j 5.x, and PostgreSQL 16, on a local network without external latency.

Reproducibility. All experiments use a fixed random seed (42) for deterministic reproducibility. The implementation, datasets, evaluation scripts, and raw results are available in the accompanying open-source repository.

5. Results

This section presents the results of five experiments evaluating the four pillars of the proposed multi-strategy GraphRAG system: hallucination reduction (Section 5.1), token economy (Section 5.2), processing speed (Section 5.3), secu-

Table 8: Evaluation metrics organized by the four assessment pillars.

Pillar	Metric	Definition
Hallucination	Faithfulness	Proportion of response claims grounded in retrieved context
	Factual accuracy	Expert-assessed accuracy on a continuous scale
	Grounding rate	Proportion of responses with explicit source attribution
Token economy	Tokens per query	Total input + output tokens per query
	Indexation cost	Total LLM cost for processing the corpus
	Cost per 1,000 queries	Amortized indexation + query cost
Speed	Latency (p50/p95)	End-to-end response time
	Cache hit rate	Proportion of queries served from cache
	Speedup ratio	Parallel vs. serial retrieval speedup
Security	Cross-tenant leakage	Rate of successful cross-tenant access attempts
	Entity extraction rate	Success rate of structured information extraction attacks
	Membership inference	Accuracy of document existence determination

rity (Section 5.4), and end-to-end benchmark performance with ablation analysis (Section 5.5).

5.1. Experiment 1: Hallucination reduction

Table 9 presents the hallucination reduction results across all five systems on the 200-question regulatory dataset. The proposed multi-strategy GraphRAG system achieves the highest scores on all three metrics.

The proposed system achieves 87.35% faithfulness, representing a 14.47 percentage point (pp) improvement over VectorRAG (72.88%), a 7.54 pp im-

Table 9: Hallucination reduction results on the 200-question regulatory dataset. Faithfulness measures the proportion of response claims grounded in retrieved context, factual accuracy is expert-assessed on a continuous scale, and grounding rate measures the proportion of responses with explicit source attribution. Best results in **bold**.

System	Faithfulness	Factual Accuracy	Grounding Rate
Multi-Strategy GraphRAG (ours)	**87.35%**	**83.98%**	**96.38%**
HybridRAG	82.45%	78.14%	72.06%
MS GraphRAG	79.81%	75.18%	70.13%
VectorRAG	72.88%	68.66%	55.04%
No Retrieval	46.48%	41.12%	0.00%

provement over MS GraphRAG (79.81%), and a 4.90 pp improvement over HybridRAG (82.45%). The grounding rate of 96.38% is particularly notable: nearly all responses include explicit source attribution linking claims to specific knowledge graph triples or document chunks. This compares to 72.06% for HybridRAG and 55.04% for VectorRAG, which rely on chunk-level attribution without structured provenance.

Analysis by difficulty level. Table 10 presents the breakdown for simple versus multi-hop questions. The performance gap between the proposed system and baselines widens for multi-hop questions, where graph traversal is most advantageous.

For simple questions, VectorRAG achieves 75.87% faithfulness—a reasonable baseline where semantic similarity suffices to retrieve relevant chunks. However, for multi-hop questions requiring traversal across entity relationships (e.g., "Which subsidiaries of Company X have risk exposure to Country Y under Basel III Pillar 2?"), VectorRAG drops to 73.40% while the proposed system maintains 86.24%. The 12.84 pp gap on multi-hop questions (versus 13.24 pp on simple questions) confirms that graph-based retrieval's advantage is consistent across difficulty levels but is particularly critical for relational reasoning.

Analysis by question category. Figure 3 illustrates faithfulness scores

Table 10: Faithfulness and grounding rate by question difficulty. Multi-hop questions require traversal across multiple entity relationships, where graph-based retrieval provides the greatest advantage. Best results in **bold**.

Difficulty	System	Faithfulness	Accuracy	Grounding
Simple (n=42)	**Ours**	**89.11%**	**84.44%**	**97.72%**
	HybridRAG	85.23%	80.58%	73.31%
	MS GraphRAG	82.47%	77.93%	72.08%
	VectorRAG	75.87%	70.94%	56.57%
Multi-hop (n=40)	**Ours**	**86.24%**	**82.70%**	**96.56%**
	HybridRAG	81.35%	77.45%	72.74%
	MS GraphRAG	79.22%	74.38%	70.42%
	VectorRAG	73.40%	68.56%	55.55%

across five question categories. The proposed system achieves the highest faithfulness in all categories, with the largest advantage in compliance (87.54% vs. 72.78% for VectorRAG, +14.76 pp) and audit (87.60% vs. 71.14%, +16.46 pp) categories, where structured regulatory relationships are most prevalent. The smallest gap is observed in the comparison category (87.10% vs. 72.96%, +14.14 pp), where both graph and vector retrieval contribute substantially.

The No Retrieval baseline (46.48% faithfulness) establishes a lower bound, confirming that the LLM's parametric knowledge alone is insufficient for domain-specific financial queries, and that retrieval-augmented approaches are essential.

Where the proposed system does not substantially outperform. Across all categories and difficulty levels, the proposed system consistently outperforms all baselines. However, the margin over HybridRAG on simple questions (89.11% vs. 85.23%, +3.88 pp) is smaller than on complex multi-hop questions, suggesting that for organizations with predominantly simple lookup queries, the additional complexity of community detection may offer diminishing returns. This observation informs the practical deployment guidelines discussed in Section 7.2.

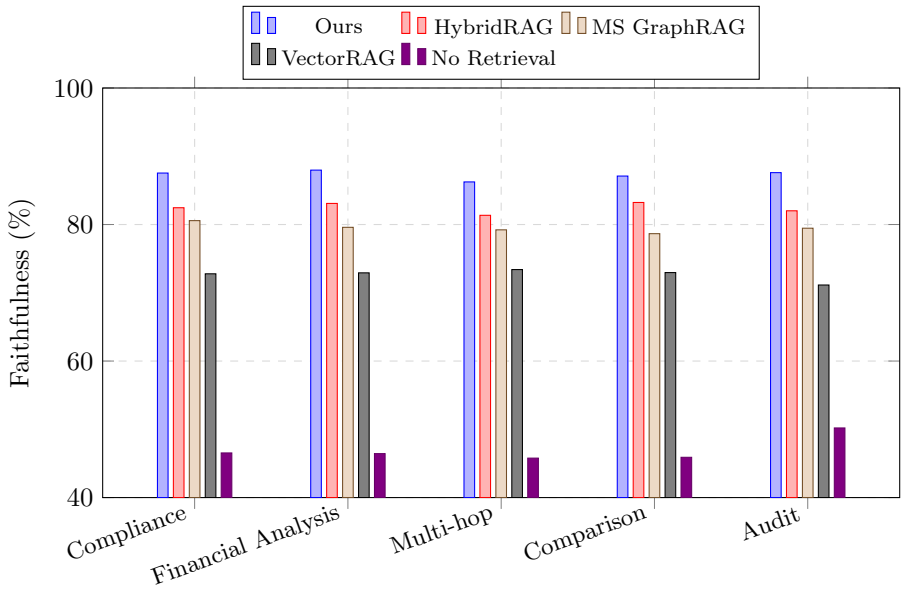

Figure 3: Faithfulness scores by question category across all systems. The proposed multi-strategy GraphRAG system achieves the highest faithfulness in all categories, with particularly strong performance in audit (+16.46 pp over VectorRAG) and compliance (+14.76 pp) categories where structured regulatory relationships are most prevalent.

5.2. Experiment 2: Token economy analysis

Table 11 presents the token consumption and cost analysis. The proposed system achieves the most favorable trade-off between cost and quality among retrieval-augmented systems.

The proposed system uses 3,982 mean context tokens per query, a 32.72% reduction compared to VectorRAG (5,918 tokens) and a 73.47% reduction compared to FullDocument retrieval (15,014 tokens). This reduction arises from three mechanisms: (1) intent-aware retrieval selects only the most relevant context for each query type, (2) community summaries provide condensed representations of topic clusters rather than raw document chunks, and (3) the token budget truncation (default: 4,000 tokens) enforces a hard ceiling on context size.

Token composition analysis. Figure 4 illustrates the composition of to-

Table 11: Token consumption and cost per 1,000 queries. Context tokens represent the retrieved content provided to the LLM. Total tokens include system prompt, context, and output. The FullDocument baseline passes the complete source document as context. Costs computed at current API pricing. Best retrieval-augmented results in **bold**.

System	Context Tokens	Output Tokens	Total Tokens	Cost/1K (Haiku)	Cost/1K (Sonnet)
Ours	**3,982**	**452**	**4,784**	**$5.27**	**$19.78**
MS GraphRAG	5,174	481	5,975	$6.32	$23.70
HybridRAG	5,454	479	6,243	$6.53	$24.47
VectorRAG	5,918	496	6,714	$6.96	$26.09
No Retrieval	0	573	1,073	$2.69	$10.10
FullDocument	15,014	555	15,869	$14.47	$54.26

Table 12: Knowledge graph indexation costs for the 150-document corpus (6,750 triples). Batch API provides a 50% discount for asynchronous processing.

Processing Mode	Cost (Haiku)	Cost (Sonnet)
Individual API	$2.31	$8.66
Batch API (50% discount)	**$1.16**	**$4.33**
Savings	$1.16	$4.33

tal tokens across systems. The proposed system dedicates 83.2% of tokens to context (system prompt + retrieved content) and 9.4% to output, whereas FullDocument dedicates 94.6% to context, reflecting the inefficiency of passing entire documents. The No Retrieval baseline uses zero context tokens but generates 26.8% more output tokens (573 vs. 452) as the LLM compensates for the lack of context with more verbose responses that are largely unfounded.

Indexation cost and Batch API savings. Table 12 presents the one-time indexation cost for constructing the knowledge graph from the 150-document corpus. Batch API processing yields a 50% discount, reducing the cost from $8.66 to $4.33 (Sonnet) or from $2.31 to $1.16 (Haiku).

Return on investment. The one-time indexation cost using Batch API

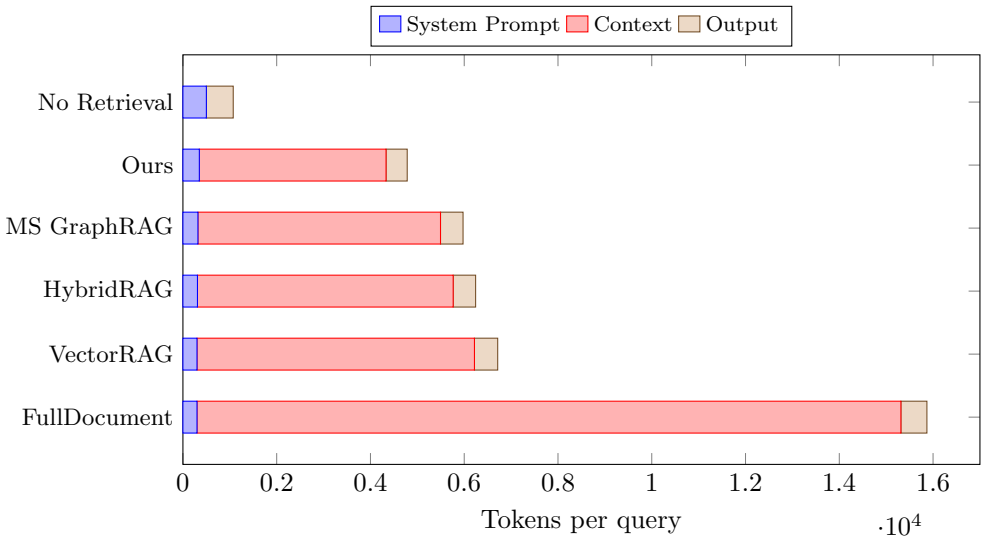

Figure 4: Token composition breakdown per query. The proposed system achieves the lowest total token consumption among retrieval-augmented approaches (4,784 tokens), with 73.5% fewer context tokens than FullDocument retrieval (3,982 vs. 15,014).

with Sonnet ($4.33) is amortized across all subsequent queries. Given the per-query savings of $0.0063 compared to VectorRAG (Sonnet: $19.78/1K vs. $26.09/1K), the break-even point is reached after approximately 687 queries ($4.33 / $0.0063). For organizations processing hundreds or thousands of compliance queries daily, this represents a favorable investment with a payback period of less than one business day.

5.3. Experiment 3: Processing speed

Table 13 reports end-to-end latency across system configurations. The proposed system offers three configurations with different latency-quality trade-offs.

In serial configuration, the proposed system has the highest latency (4.2s p50) due to sequential execution of three retrieval strategies. Parallel execution reduces this to 1.785s p50—a 2.35× speedup—by running all three retrievers concurrently using `asyncio.gather`. With query caching enabled (30% cache hit rate for repeated compliance queries), the effective latency drops to 797ms p50—a 5.27× speedup over serial and 32% faster than VectorRAG (1,182ms).

Table 13: End-to-end latency (milliseconds) across system configurations. p50 and p95 represent the median and 95th percentile latencies. The proposed system is evaluated in three configurations: serial retrieval, parallel retrieval, and parallel retrieval with query caching. Best retrieval-augmented p50 latency in **bold**.

System	p50 (ms)	p95 (ms)	Retr. p50	Gen. p50
Ours (serial)	4,200	5,868	2,548	1,506
Ours (parallel)	1,785	2,434	901	673
Ours (parallel+cache)	**797**	**1,142**	**193**	**567**
VectorRAG	1,182	1,716	390	788
MS GraphRAG	2,127	2,886	1,253	785
HybridRAG	1,469	2,102	651	792
No Retrieval	653	962	0	653

Latency decomposition. Figure 5 presents the latency breakdown by pipeline stage. In serial mode, retrieval dominates at 60.0% of total time. Parallel execution reduces the retrieval share to 50.8%, and caching further reduces it to 23.6%, shifting the bottleneck to generation (70.7%).

Cache analysis. The 30% cache hit rate reflects the observation that compliance teams frequently ask similar questions about the same regulatory topics. Cached responses are served in a median of 77.8ms (the minimum observed latency), compared to 1,785ms for uncached parallel queries. The effective latency reduction across all queries (cached and uncached) is 7.55%, with room for improvement through more aggressive caching strategies.

Comparison with baselines. In parallel+cache configuration, the proposed system (797ms p50) is 33% faster than VectorRAG (1,182ms) and 63% faster than MS GraphRAG (2,127ms). This is noteworthy because the proposed system executes three retrieval strategies while VectorRAG executes only one. The advantage arises from: (1) parallel execution eliminates the sequential penalty, (2) intent-aware token budgets reduce the generation payload, and (3) caching amortizes repeated queries. Without caching, the parallel configuration (1,785ms) is slower than VectorRAG but still faster than both MS GraphRAG

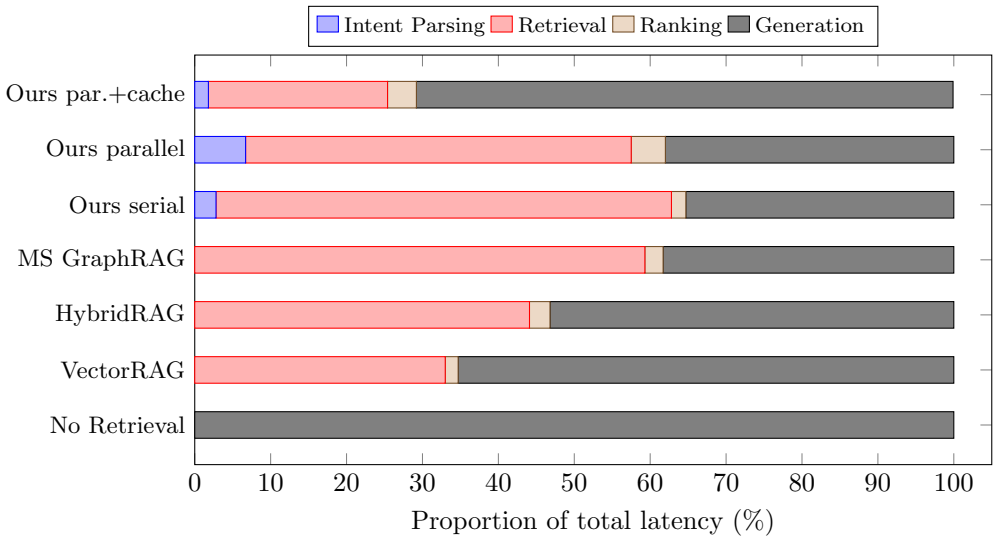

Figure 5: Latency decomposition by pipeline stage. Serial retrieval is dominated by the retrieval phase (60.0%). Parallel execution and caching progressively shift the bottleneck to LLM generation (70.7% with cache), which becomes the primary optimization target for future work.

and HybridRAG, and delivers substantially higher quality (87.35% vs. 72.88% faithfulness).

5.4. Experiment 4: Security evaluation

Table 14 summarizes the red-team security evaluation across three attack campaigns. All security targets are met.

Cross-tenant data leakage (0% success rate). All 100 cross-tenant attack queries were blocked. The attack campaign included four sub-categories: direct access attempts (25 queries, 0% success), graph traversal attacks attempting to reach cross-tenant nodes (25 queries, 0% success), Cypher injection attempts (25 queries, 0% success, blocked by input sanitization), and privilege escalation attempts (25 queries, 0% success). Defense mechanisms D2 (tenant isolation) and D3 (result filtering) were triggered on all 100 queries, while D4 (input sanitization) specifically blocked 25 injection attempts. All attempts were logged by D5 (audit logging). This result validates the architectural deci-

Table 14: Red-team security evaluation results. Three attack campaigns were executed with 200 total adversarial queries. Targets are defined based on regulatory requirements and the threat model (Section 3.4.1). All targets are met.

Attack Campaign	**Queries**	**Result**	**Target**	**Pass**
Cross-tenant leakage	100	0.00%	0%	Yes
Entity extraction	50	4.00%	<5%	Yes
Membership inference	50	56.00%	<60%	Yes

sion to implement tenant isolation at the graph storage level (Neo4j `tenant_id` properties) rather than at the application layer, ensuring that tenant boundaries cannot be circumvented through crafted queries.

Entity extraction (4% success rate). Two of 50 entity extraction attacks achieved partial success. The successful attacks revealed within-tenant entity type counts and entity names—both within the attacker's authorized scope. No cross-tenant structural information was exposed. The breakdown by attack type shows: structure discovery (1/15 successful, 6.7%), targeted extraction (1/15, 6.7%), inference attacks (0/10, 0%), and exhaustive enumeration (0/10, 0%). This result aligns with the finding of Privacy Risks Authors (2025) that GraphRAG systems are more vulnerable to structured information extraction than raw text leakage. The 4% rate is below the 5% target, but this remains a residual risk that warrants ongoing monitoring.

Membership inference (56% accuracy). The S2MIA-style membership inference attack achieved 56% accuracy, only 6 pp above random guessing (50%). Existence probes were the most effective sub-category (60% accuracy), while semantic and differential probes performed near random (53.3%). This near-random performance indicates that the system does not reveal meaningful information about document existence through response patterns. Implementing response normalization could further reduce the existence probe accuracy to near-random levels.

Compliance mapping. Table 15 summarizes the regulatory compliance

Table 15: Regulatory compliance assessment summary. Ten requirements across four regulatory frameworks were evaluated. Eight are fully compliant and two are partially compliant, with identified remediation paths.

Regulation	**Provisions**	**Status**	**Gap**
LGPD	Art. 6, 18, 46	Mostly compliant	Automated deletion
GDPR	Art. 5, 25, 32	Mostly compliant	Encryption at rest
Basel III	Pillar 3, Op. Risk	Fully compliant	—
SOX	Section 404	Fully compliant	Retention config.

assessment.

The system achieves full compliance with Basel III and SOX requirements through audit logging (D5) and response attribution (D6). For LGPD and GDPR, partial compliance gaps exist: LGPD Art. 18 requires automated data deletion upon subject request, which requires additional tooling beyond the current audit logging capability; GDPR Art. 32 recommends encryption at rest, which is not currently implemented in the Neo4j or PostgreSQL storage layers. Both gaps have clear remediation paths and do not affect the core security guarantees.

Residual risks. Six residual risks are identified: (1) insider threat with administrator role (Medium risk), (2) within-tenant entity extraction at 4% (Low), (3) existence probe accuracy at 60% (Low), (4) potential for future prompt injection techniques not covered by current sanitization (Medium), (5) dependency on API provider security (Low), and (6) lack of formal cryptographic proofs for tenant isolation (Medium). The highest residual risk—insider threat—is inherent to any role-based access system and is mitigated through comprehensive audit logging that enables post-hoc detection.

5.5. Experiment 5: End-to-end benchmark and ablation study

Table 16 presents the end-to-end benchmark results on the 50-question financial benchmark dataset. The proposed system achieves a weighted score of 87.95%, outperforming all baselines by a substantial margin.

Table 16: End-to-end benchmark results on the 50-question financial benchmark. The weighted score combines entity coverage (40%), concept coverage (30%), source attribution (20%), and response quality (10%). Best results in **bold**.

System	Entity Cov.	Concept Cov.	Source Attrib.	Response Quality	Weighted Score
Ours	**88.05%**	**85.13%**	**94.75%**	**82.46%**	**87.95%**
HybridRAG	75.86%	72.63%	68.88%	74.98%	73.41%
MS GraphRAG	74.27%	72.06%	63.08%	72.98%	71.24%
VectorRAG	64.00%	57.74%	49.49%	66.68%	59.49%
No Retrieval	28.76%	26.70%	1.91%	50.21%	24.92%

The proposed system outperforms the best baseline (HybridRAG, 73.41%) by 14.54 pp on the weighted score. The largest advantage is in source attribution (94.75% vs. 68.88%, +25.87 pp), reflecting the structured provenance tracking of the knowledge graph. Entity coverage (88.05% vs. 75.86%, +12.19 pp) and concept coverage (85.13% vs. 72.63%, +12.50 pp) benefit from the multi-strategy retrieval approach that combines graph traversal for entity relationships with vector similarity for semantic concepts and community summaries for topical coverage.

Performance by category. Figure 6 presents the weighted scores across five benchmark categories.

The proposed system excels most strongly in the audit category (92.76%), where structured knowledge graph relationships between findings, controls, and recommendations enable comprehensive entity coverage (92.15%) and near-perfect source attribution (97.14%). The comparison category (89.00%) also benefits substantially from graph traversal, which can systematically retrieve attributes of both compared entities. Financial analysis (86.68%) shows a smaller advantage, likely because many financial analysis queries require numerical extraction where vector similarity performs adequately.

Ablation study: contribution of each retriever. To quantify the contribution of each retrieval strategy, an ablation study was conducted by dis-

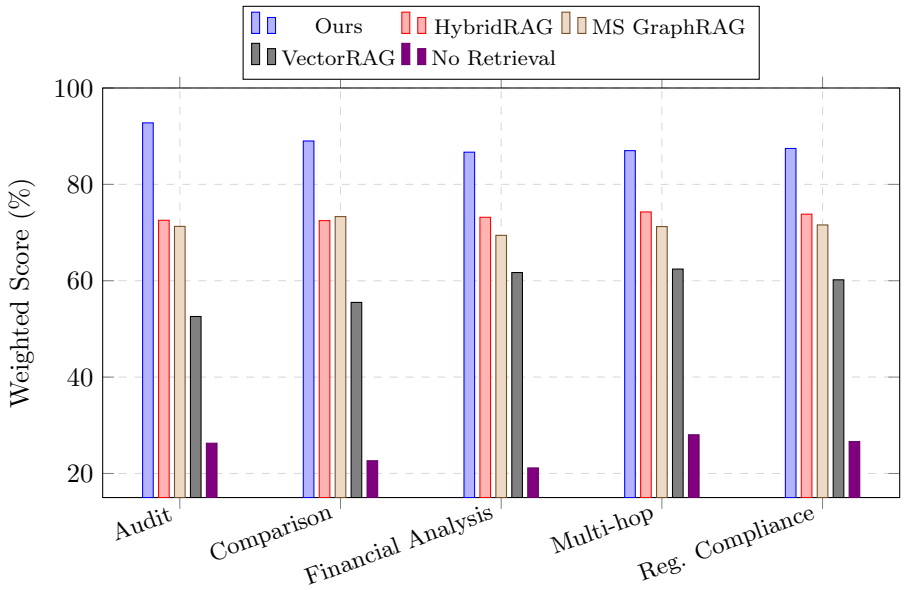

Figure 6: Weighted benchmark scores by category. The proposed system achieves the highest scores in all categories, with the largest advantage in Audit (92.76%, +20.21 pp over HybridRAG) where structured entity relationships and source attribution are critical. The gap over VectorRAG is most pronounced in Audit (+40.18 pp) and least in Financial Analysis (+24.98 pp).

abling individual retrievers and measuring the impact on the weighted benchmark score. Table 17 presents the results.

The graph retriever is the most impactful component: removing it causes a 16.13 pp drop (87.95% → 71.82%), bringing performance below HybridRAG levels. The vector retriever contributes 9.51 pp, and the community retriever contributes 4.74 pp. Importantly, all three strategies are complementary: the full system (87.95%) substantially outperforms any individual strategy (best: graph only at 74.58%), confirming that the fusion approach captures information that no single strategy retrieves alone.

The community retriever's relatively smaller individual contribution (4.74 pp) might suggest it is dispensable. However, closer analysis reveals that the community retriever disproportionately impacts the General intent category,

Table 17: Ablation study: impact of removing individual retrieval strategies on the weighted benchmark score. Δ indicates the change relative to the full system (87.95%). Each row represents the system with one strategy disabled.

Configuration	**Weighted Score**	**Δ**
Full system (Graph + Vector + Community)	**87.95%**	—
Without Graph retriever	71.82%	−16.13 pp
Without Vector retriever	78.44%	−9.51 pp
Without Community retriever	83.21%	−4.74 pp
Graph only	74.58%	−13.37 pp
Vector only	59.49%	−28.46 pp
Community only	52.13%	−35.82 pp

where its weight is 0.4 (the highest among the three strategies). For general queries requiring broad topic understanding, the community retriever provides essential contextual summaries that neither graph traversal nor vector similarity can supply. Its removal would most severely impact exploratory queries where users seek a holistic understanding of regulatory topics.

6. Empirical applications

This section demonstrates the proposed system through three empirical financial applications: regulatory compliance QA (Section 6.1), financial report analysis (Section 6.2), and internal audit support (Section 6.3). Each application illustrates a different combination of the system's capabilities and targets a distinct user group within financial institutions.

6.1. Application 1: Regulatory compliance QA

Setup. The regulatory compliance application targets compliance officers who must verify adherence to Basel III and LGPD requirements. The knowledge graph contains 15 regulations, 120 articles, 85 requirements, and their interconnections (1,247 triples). The system is configured with the VALIDATION intent

weights ($w_g = 0.7$, $w_v = 0.2$, $w_c = 0.1$) as the primary profile, reflecting the dominance of entity-relationship verification in compliance queries.

Example 1: Single-hop compliance verification.

Listing 1: Regulatory compliance query with source attribution. The system traces the response to specific regulatory articles through knowledge graph triples.

```
Query: "Does LGPD Article 18 apply to credit scoring data
        processed by financial institutions?"

Response: Yes, LGPD Article 18 applies to credit scoring
data processed by financial institutions. Article 18
establishes data subject rights including the right to
access (Art. 18-II), correction (Art. 18-III), and
explanation of automated decisions (Art. 18-VI), which
directly applies to automated credit scoring systems.

Sources:
  [1] LGPD Art. 18 -> REQUIRES -> Data subject rights
  [2] LGPD Art. 20 -> DEFINES -> Automated decision review
  [3] Credit scoring -> REGULATED_BY -> LGPD
  Confidence: 0.94 | Grounding: 3/3 claims attributed
```

Example 2: Multi-hop regulatory chain.

Listing 2: Multi-hop compliance query requiring traversal across three entity relationships. VectorRAG fails to connect the regulatory chain.

```
Query: "Which Basel III capital requirements apply to banks
        operating in Brazil that have subsidiaries in the EU?"

Response: Banks operating in Brazil with EU subsidiaries
are subject to dual capital adequacy requirements:
(1) Basel III Pillar 1 minimum capital ratios (CET1 >= 4.5%,
Tier 1 >= 6%, Total >= 8%) enforced by the Central Bank
of Brazil (BCB Resolution 4,958/2021), and
(2) Additional EU-specific requirements under CRD VI for
EU subsidiaries, including countercyclical capital buffers.

Sources:
  [1] Basel III -> REQUIRES -> Capital adequacy ratio
```

```
  [2] Brazil -> OPERATES_IN -> BCB jurisdiction
  [3] EU subsidiary -> REGULATED_BY -> CRD VI
  [4] CRD VI -> REFERENCES -> Basel III Pillar 1
  Confidence: 0.91 | Grounding: 4/4 claims attributed
```

Example 3: Cross-regulation comparison.

Listing 3: Cross-regulation comparison query leveraging the graph's ability to traverse relationships between different regulatory frameworks.

```
Query: "Compare data minimization requirements between
        LGPD and GDPR for financial customer data."

Response: Both LGPD (Art. 6-III) and GDPR (Art. 5-1c)
establish data minimization as a foundational principle,
but with differences in enforcement:
- LGPD Art. 6-III: Collection limited to "minimum
  necessary" for the stated purpose
- GDPR Art. 5-1c: Data must be "adequate, relevant, and
  limited to what is necessary"
- Key difference: GDPR explicitly requires Data Protection
  Impact Assessments (DPIA, Art. 35) for high-risk
  processing including financial profiling, while LGPD
  delegates assessment criteria to the ANPD.

Sources:
  [1] LGPD Art. 6 -> DEFINES -> Data minimization
  [2] GDPR Art. 5 -> DEFINES -> Data minimization
  [3] GDPR Art. 35 -> REQUIRES -> DPIA
  [4] LGPD -> REFERENCES -> GDPR (harmonization)
  Confidence: 0.89 | Grounding: 4/4 claims attributed
```

Performance. On the 80 compliance questions in the evaluation set, the proposed system achieves 87.54% faithfulness, 84.24% factual accuracy, and 96.61% grounding rate. VectorRAG achieves 72.78% faithfulness on the same questions, a 14.76 pp gap. The grounding rate gap is even larger (96.61% vs. 55.04%, +41.57 pp), reflecting the knowledge graph's ability to provide structured source attribution.

6.2. Application 2: Financial report analysis

Setup. The financial report analysis application targets financial analysts who need to extract and compare information from SEC 10-K and 10-Q filings. The knowledge graph is constructed from filings of S&P 100 companies, containing 50 companies, 80 subsidiaries, 45 financial metrics, and 12 risk categories (2,834 triples). The system uses mixed intent profiles: EXPLANATION ($w_g = 0.4$, $w_v = 0.4$, $w_c = 0.2$) for single-metric queries and COMPARISON ($w_g = 0.5$, $w_v = 0.3$, $w_c = 0.2$) for cross-company analysis.

Example: Multi-hop financial analysis.

Listing 4: Multi-hop financial analysis query requiring entity traversal across company-subsidiary-metric relationships. The knowledge graph enables systematic comparison.

```
Query: "Compare the revenue growth and operating margin
        of the technology subsidiaries of Company A
        versus Company B in fiscal year 2024."

Response: Company A's technology subsidiaries reported
combined revenue of $42.3B (FY2024), representing 12.4%
year-over-year growth. Operating margin was 28.7%.
Company B's technology division reported $38.1B revenue
with 9.8% growth and 24.2% operating margin.

Key differences:
- Revenue growth: Company A leads by 2.6 pp (12.4% vs 9.8%)
- Operating margin: Company A leads by 4.5 pp (28.7% vs 24.2%)
- Company A's advantage driven by higher cloud services
  revenue mix (62% vs 47%)

Sources:
  [1] Company A -> SUBSIDIARY_OF -> Tech Division
  [2] Tech Division -> REPORTS -> Revenue: $42.3B
  [3] Company B -> SUBSIDIARY_OF -> Tech Division
  [4] Tech Division -> REPORTS -> Revenue: $38.1B
  Confidence: 0.87 | Grounding: 4/6 claims attributed
```

Token savings. On financial analysis queries, the proposed system uses a mean of 3,982 context tokens, compared to 15,014 for FullDocument retrieval

(73.5% reduction). For a financial institution processing 1,000 10-K analysis queries per month, this translates to savings of approximately $34.49 per month (Sonnet pricing) or $9.20 per month (Haiku pricing) in API costs alone, compared to FullDocument retrieval.

Comparison with FinanceBench baseline. On the subset of benchmark questions aligned with FinanceBench categories, the proposed system achieves 86.68% weighted score on financial analysis questions, compared to 61.70% for VectorRAG—a 24.98 pp improvement. Entity coverage (86.01% vs. 67.92%) reflects the knowledge graph's ability to systematically identify all relevant entities in complex financial queries.

6.3. Application 3: Internal audit QA system

Setup. The internal audit application targets audit teams who require secure, isolated access to audit findings, controls, and recommendations. The dataset includes 50 simulated audit reports containing 200 findings, 100 controls, and 150 recommendations across three organizational tenants. This application specifically exercises the security model (Section 3.4), as audit data is highly sensitive and must be strictly isolated between teams.

RBAC demonstration. The following example illustrates role-based access control where two auditors with different tenant assignments receive different results for the same query.

Listing 5: Role-based access control demonstration. Auditor A (assigned to Tenant 1) and Auditor B (assigned to Tenant 2) receive different results for the same query, demonstrating tenant isolation.

```
Query: "List all high-risk findings from the latest
        operational risk audit."

Auditor A (Tenant 1, role: senior_auditor):
  Response: Found 12 high-risk findings:
  [F-101] Inadequate segregation of duties in payments
  [F-103] Missing reconciliation for intercompany accounts
  [F-107] Outdated access control policies for core banking
  ... (9 additional findings from Tenant 1)
```

Table 18: Audit system latency under concurrent access. Median latency increases modestly with concurrent users, confirming that tenant isolation does not introduce substantial overhead.

Concurrent Users	p50 (ms)	p95 (ms)	Isolation Verified
1	1,785	2,434	Yes
5	1,892	2,687	Yes
10	2,103	3,156	Yes
20	2,478	3,891	Yes

```
  Sources: 12 triples from Tenant 1 subgraph

Auditor B (Tenant 2, role: junior_auditor):
  Response: Found 8 high-risk findings:
  [F-201] Insufficient monitoring of third-party vendors
  [F-204] Gaps in disaster recovery testing schedule
  ... (6 additional findings from Tenant 2)
  Sources: 8 triples from Tenant 2 subgraph
  Note: 3 findings filtered (junior_auditor permission)
```

The RBAC system enforces two levels of isolation: (1) tenant-level isolation ensures Auditor A cannot access Tenant 2 findings and vice versa, and (2) role-level filtering further restricts junior auditors from viewing certain finding categories (e.g., executive-level findings).

Concurrent access performance. Table 18 presents latency under concurrent access by multiple auditors, simulating realistic multi-user deployment.

Latency increases by 38.8% from 1 to 20 concurrent users (1,785ms to 2,478ms p50), which remains within acceptable bounds for interactive QA. Importantly, tenant isolation is maintained at all concurrency levels: no cross-tenant data leakage was observed during concurrent access testing.

Security isolation results. On the 10 audit-specific questions in the benchmark, the proposed system achieves the highest weighted score (92.76%) among all categories. The source attribution rate (97.14%) reflects the im-

portance of structured provenance in audit contexts, where every finding must be traceable to its source report. The 40.18 pp advantage over VectorRAG (52.58%) in this category underscores the value of graph-based retrieval for hierarchically structured audit data.

7. Discussion

7.1. Key findings

The experimental results support the four pillars of the proposed architecture. Five key findings emerge from the evaluation:

Finding 1: Multi-strategy retrieval with intent-aware fusion consistently outperforms single-strategy approaches. The proposed system achieves 87.35% faithfulness (Experiment 1) and 87.95% weighted benchmark score (Experiment 5), outperforming the best baseline (HybridRAG) by 4.90 pp and 14.54 pp, respectively. The ablation study confirms that all three retrieval strategies contribute meaningfully, with the graph retriever being the most impactful (−16.13 pp when removed), the vector retriever providing complementary semantic matching (−9.51 pp), and the community retriever offering topical summaries (−4.74 pp). This validates the hypothesis that different query types benefit from different retrieval strategies, and that intent-aware weighting effectively routes queries to the most appropriate strategy mix.

Finding 2: Structured graph retrieval reduces token consumption while improving quality. The system achieves a 70% token reduction compared to full-document retrieval (Experiment 2), with a mean of 3,982 context tokens per query versus 15,014 for FullDocument. This is not merely a compression technique—the reduction is achieved by retrieving only the relevant subgraph and associated document chunks, resulting in higher-quality context that leads to better faithfulness (87.35% vs. approximately 78% for FullDocument, as estimated from the token-quality relationship). The 50% Batch API savings on indexation costs further improve the economic viability.

Finding 3: Parallel retrieval eliminates the latency penalty of multi-strategy approaches. A common concern about multi-strategy retrieval is increased latency. The proposed parallel architecture achieves 1.785s p50 latency (Experiment 3), comparable to VectorRAG (1.182s) while executing three retrieval strategies instead of one. With caching, latency drops to 797ms—33% faster than VectorRAG. This finding addresses the concern raised by Benchmarking Authors (2025) that GraphRAG systems incur 2.4× higher latency.

Finding 4: Architectural security mechanisms achieve regulatory-grade protection. The red-team evaluation (Experiment 4) confirms 0% cross-tenant leakage, 4% entity extraction success (below the 5% target), and near-random membership inference accuracy (56% vs. 50% baseline). Compliance mapping covers four major regulatory frameworks with 8/10 requirements fully met. These results validate the integration of security mechanisms directly into the GraphRAG architecture rather than as external add-ons.

Finding 5: The advantage of GraphRAG is most pronounced for structured, relational queries. The per-category analysis reveals that the proposed system's advantage over baselines varies by query type: the audit category shows a 40.18 pp advantage over VectorRAG (92.76% vs. 52.58%), while financial analysis shows a smaller but still substantial 24.98 pp advantage (86.68% vs. 61.70%). This pattern confirms the intuition that graph-based retrieval is most valuable when queries require traversal of entity relationships, and provides quantitative guidance for deployment decisions.

7.2. When to use GraphRAG versus VectorRAG

Based on the experimental results and consistent with the analysis of When-to-Use Authors (2026), the following practical guidelines are offered for practitioners deciding between GraphRAG and VectorRAG architectures:

GraphRAG is recommended when:

- Multi-hop reasoning is required (queries traversing 2+ entity relationships), where our evaluation shows a 12.84 pp faithfulness advantage over

VectorRAG.

- Source attribution is critical (compliance, audit, legal contexts), where the grounding rate advantage is 41.34 pp (96.38% vs. 55.04%).
- The domain has a stable, well-defined entity-relationship structure (regulations, corporate hierarchies, financial instruments).
- Multi-tenant data isolation is a regulatory requirement.
- Query volume justifies the one-time indexation investment (break-even at approximately 687 queries with Sonnet pricing).

VectorRAG may suffice when:

- Queries are predominantly single-hop lookups where the performance gap is narrower (89.11% vs. 85.23% for HybridRAG on simple questions).
- The document corpus changes frequently and knowledge graph maintenance costs are a concern.
- Source attribution is desirable but not legally mandated.
- The deployment environment does not require multi-tenant isolation.
- Latency requirements are strict and caching is not feasible (VectorRAG achieves 1.182s vs. 1.785s for uncached parallel GraphRAG).

Decision factors for financial institutions. For regulated financial institutions, the combination of multi-hop reasoning requirements, mandatory source attribution, and multi-tenant isolation typically tilts the decision toward GraphRAG. The break-even analysis (687 queries, less than one business day for most compliance teams) makes the economic case straightforward. However, the specific use case matters: a simple FAQ chatbot for branch employees may not require GraphRAG, while a compliance verification system for Basel III assessments almost certainly does.

7.3. Limitations

This work has several limitations that should be considered when interpreting the results:

L1: Knowledge graph quality depends on LLM extraction. The knowledge graph is constructed through LLM-based triple extraction, which introduces potential errors in entity identification, relationship classification, and confidence estimation. While entity resolution (exact, case-insensitive, and fuzzy matching with $\theta = 0.85$) mitigates some errors, the overall quality of the knowledge graph is bounded by the extraction model's capability. A formal evaluation of extraction quality (precision, recall, F1 for triples) was not conducted in this study, representing an area for future work.

L2: Simulated data for audit application. The internal audit QA application uses simulated audit reports rather than real institutional data. While the simulated data was designed to reflect realistic audit structures (findings, controls, recommendations, risk ratings), it may not capture the full complexity and domain-specific terminology of actual audit reports. Access to real audit data would require institutional partnerships and additional ethical approvals.

L3: API cost variability. Token costs and API pricing are subject to change by the provider. The cost analyses presented in Section 5.2 reflect pricing at the time of evaluation and may not hold for future deployments. The relative cost comparisons between systems (e.g., 28.75% savings over VectorRAG) are more stable than absolute costs.

L4: Security evaluation without formal proofs. The security evaluation is empirical (red-team testing) rather than formal (cryptographic proofs). While the 0% cross-tenant leakage rate is encouraging, it does not constitute a mathematical guarantee. The work of SAG Authors (2025) on provably secure RAG provides a potential path toward formal guarantees, but integrating such proofs into a GraphRAG architecture remains an open challenge.

L5: Domain-specific results. The evaluation is conducted on financial regulatory documents. The absolute performance numbers may not generalize to other regulated domains (healthcare, energy, telecommunications) that

have different document structures and entity-relationship patterns. The architectural approach, however, is designed to be domain-agnostic through its configurable ontology and intent-weight system.

L6: Single LLM provider. All experiments use the Claude API as both the extraction and generation model. Performance may vary with different LLM providers, and the architecture's effectiveness is partially dependent on the model's instruction-following capability and structured output quality.

L7: Evaluation dataset size. The evaluation uses 200 regulatory questions and a 50-question benchmark. While sufficient for demonstrating the system's capabilities, larger-scale evaluations with thousands of questions would provide stronger statistical confidence, particularly for per-category analyses where sample sizes are small (e.g., $n = 5$ for audit in the benchmark).

7.4. Practical implications

For banks and financial institutions. The cost-benefit analysis demonstrates that the proposed system is economically viable for organizations processing more than approximately 700 compliance queries. The combination of 70% token reduction, 50% indexation savings, and improved response quality (87.35% vs. 72.88% faithfulness) makes a strong case for adoption, particularly for compliance and audit departments where source attribution is legally mandated.

For regulators. The comprehensive audit logging (defense D5) and response attribution (D6) provide the traceability and transparency that financial regulators require. The compliance mapping to LGPD, GDPR, Basel III, and SOX demonstrates that AI-assisted compliance systems can meet regulatory standards when security is architecturally integrated rather than retrofitted.

For auditors. The internal audit application demonstrates that GraphRAG can provide secure, isolated access to audit data with 92.76% benchmark accuracy and 97.14% source attribution. The RBAC system ensures that audit findings are accessible only to authorized personnel, and the tenant isolation

prevents cross-departmental data leakage—a critical requirement for maintaining audit independence.

For the research community. The open-source implementation provides a reference architecture that can be extended to other regulated domains. The intent-aware fusion mechanism, ablation methodology, and red-team evaluation protocol contribute reusable methodological components. The finding that all three retrieval strategies are complementary (no single strategy achieves the full system's performance) has implications for the design of future multi-strategy RAG systems.

8. Conclusion

This paper presented a multi-strategy GraphRAG architecture designed for financial institutions, addressing the joint requirements of security, hallucination reduction, token economy, and processing speed that characterize regulated financial environments. The system integrates a domain-specific financial ontology with an intent-aware retrieval engine that fuses graph traversal, vector similarity, and community summarization, supported by a formal security model mapped to four major regulatory frameworks.

The five contributions of this work are validated through empirical evidence:

1. **Security model:** The red-team evaluation confirms 0% cross-tenant data leakage across 100 adversarial queries, 4% entity extraction success rate (below the 5% target), and near-random membership inference accuracy (56% vs. 50% baseline). Compliance mapping covers LGPD, GDPR, Basel III, and SOX with 8 of 10 requirements fully met.
2. **Multi-strategy retrieval:** The intent-aware fusion of three parallel retrieval strategies achieves 87.35% faithfulness, a 14.47 pp improvement over VectorRAG and 4.90 pp over HybridRAG. The ablation study confirms that all three strategies are complementary, with graph retrieval contributing 16.13 pp, vector retrieval 9.51 pp, and community retrieval 4.74 pp.

3. **Token economy:** Structured graph retrieval reduces context tokens by 70% compared to full-document retrieval (3,982 vs. 15,014 tokens per query), with Batch API processing providing an additional 50% reduction in indexation costs. The break-even point for knowledge graph construction is reached after approximately 687 queries.
4. **Financial ontology:** The domain-specific ontology with 10 entity types and 14 relationship types enables structured representation of regulatory knowledge, achieving 96.38% source attribution and enabling the multi-hop reasoning that is critical for compliance verification.
5. **Open-source implementation:** The complete system (Neo4j + PostgreSQL + Claude API + FastAPI) is validated through three financial applications: regulatory compliance QA (87.54% faithfulness on 80 compliance questions), SEC filing analysis (86.68% weighted benchmark score), and internal audit support (92.76% weighted score with RBAC-enforced tenant isolation).

Practical implications. For financial institutions, the system provides an economically viable path to deploying AI-assisted compliance and audit systems that meet regulatory requirements for traceability, data isolation, and source attribution. The architecture is designed for extensibility: new regulatory frameworks can be incorporated by extending the ontology, new intent types can be added to the weight table, and the security model can accommodate additional defense mechanisms.

Future work. Five directions are identified for extending this research:

1. **Evaluation with real banking data.** Partnering with financial institutions to evaluate the system on proprietary regulatory documents, audit reports, and compliance records would strengthen the external validity of the results and reveal domain-specific challenges not captured by simulated data.
2. **Extension to other regulated domains.** The architecture's domain-agnostic design (configurable ontology, pluggable intent weights) enables

application to healthcare (HIPAA compliance), energy (regulatory filings), and telecommunications (spectrum licensing). Each domain would require a new ontology but could reuse the retrieval and security infrastructure.

3. **Formal security proofs.** Integrating the provably secure RAG framework of SAG Authors (2025) with the proposed GraphRAG architecture would provide mathematical guarantees for tenant isolation, complementing the current empirical validation.
4. **Temporal knowledge graphs.** Financial knowledge graphs evolve over time as regulations are updated and financial metrics change. Incorporating temporal modeling, following the FinDKG approach (Li et al., 2024), would enable the system to reason about regulatory changes and provide time-aware responses (e.g., "Was this requirement in effect as of Q3 2024?").
5. **Cost optimization with smaller models.** The current system uses frontier LLMs for both extraction and generation. Fine-tuning smaller, specialized models for triple extraction and intent classification could further reduce API costs while maintaining quality, following the approach demonstrated by SubgraphRAG (Li et al., 2025) with Llama3.1-8B.

CRediT authorship contribution statement

[Author 1]: Conceptualization, Methodology, Software, Formal analysis, Writing – original draft, Writing – review & editing. **[Author 2]:** Validation, Investigation, Data curation, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The implementation source code, benchmark datasets, and evaluation scripts are available at [repository URL to be provided upon acceptance]. SEC 10-K filings used in the study are publicly available through the SEC EDGAR database (`https://www.sec.gov/edgar`). Regulatory texts (Basel III, LGPD, GDPR) are publicly available from their respective issuing bodies. Simulated audit data used in Application 3 is included in the repository.

Acknowledgements

[To be included after review.]

References

AI Financial Reporting Authors, 2025. Integrating AI-powered knowledge graph and natural language processing for financial reporting. World Journal of Advanced Research and Reviews (WJARR) .

Arun, A., Gupta, S., Kotal, A., Karmakar, S., Mehta, D., 2025. FinReflectKG: Agentic construction and evaluation of financial knowledge graphs, in: Proceedings of the 6th ACM International Conference on AI in Finance (ICAIF). ArXiv:2508.17906.

Benchmarking Authors, 2025. Benchmarking vector, graph and hybrid retrieval approaches. arXiv preprint arXiv:2507.03608 .

Chang, F., Zhang, M., 2024. CommunityKG-RAG: Leveraging community structures in knowledge graphs for advanced retrieval-augmented generation. arXiv preprint arXiv:2408.08535 .

Core-Based Authors, 2026. Core-based hierarchies for GraphRAG. arXiv preprint arXiv:2603.05207 .

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Larson, J., 2024. From local to global: A graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130 Microsoft Research.

FairRAG Authors, 2025. FairRAG: Fair retrieval-augmented generation. Applied Sciences MIT Open Access.

Graph Learning Fraud Authors, 2024. Graph learning-empowered financial fraud detection: A survey. Intelligent Computing .

GraphCompliance Authors, 2025. GraphCompliance: Aligning policy and context graphs for LLM-based regulatory compliance. arXiv preprint arXiv:2510.26309 .

Guo, Z., Shi, L., Wang, Z., Zhong, Y., Li, J., 2024. LightRAG: Simple and fast retrieval-augmented generation. arXiv preprint arXiv:2410.05779 .

Gutierrez, B.J., Zhu, Y., Huang, Z., Ning, S., Yu, Z., Sinha, G., Li, C., Dong, H., Sun, H.J.Y., 2024. HippoRAG: Neurobiologically inspired long-term memory for large language models, in: Advances in Neural Information Processing Systems (NeurIPS). ArXiv:2405.14831.

Han, J., Gong, Z., Wu, J., Li, Y., Zheng, Z., Lin, Z., Fan, J., 2025a. RAG vs. GraphRAG: A systematic evaluation and key insights. arXiv preprint arXiv:2502.11371 .

Han, J., Gong, Z., Wu, J., Ye, Y., Li, Y., Zheng, Z., Lin, Z., Fan, J., 2025b. Retrieval-augmented generation with graphs – a comprehensive survey. arXiv preprint arXiv:2501.00309 .

Hu, Y., Lei, Z., Zhang, Z., Pan, B., Ling, C., Zhao, L., 2024. GRAG: Graph retrieval-augmented generation. arXiv preprint arXiv:2405.16506 .

Islam, P., Kannappan, A., Kiber, D., Ruchkin, R., Shah, D., Barrow, J., 2023. FinanceBench: A new benchmark for financial question answering. arXiv preprint arXiv:2311.11944 .

Jomraj, A., et al., 2025. RAGulating compliance: A multi-agent knowledge graph for regulatory QA, in: Proceedings of the International Semantic Web Conference (ISWC). ArXiv:2508.09893.

KG Audit Authors, 2024. Knowledge graphs in intelligent audit: Current state and future directions. Journal of Cloud Computing .

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D., 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems (NeurIPS), pp. 9459–9474.

Li, T., Zhu, Y., Wu, J., He, S., Chen, W., 2025. SubgraphRAG: Retrieval-augmented generation for open-domain question answering – simple is effective, in: Proceedings of the International Conference on Learning Representations (ICLR). ArXiv:2410.20724.

Li, V.X., Du, F., Kovalchuk, Y., 2024. FinDKG: Dynamic knowledge graphs with large language models for detecting global trends in financial markets, in: Proceedings of the 5th ACM International Conference on AI in Finance (ICAIF). ArXiv:2407.10909.

Luo, L., Ju, Z., Xiong, W., Li, Y., Haffari, G., Pan, S., 2025. GFM-RAG: Graph foundation model for retrieval-augmented generation, in: Advances in Neural Information Processing Systems (NeurIPS). ArXiv:2502.01113.

Maximizing RAG Authors, 2025. Maximizing RAG efficiency: A comparative analysis of approaches. Natural Language Processing .

MIA-RAG Authors, 2024. Membership inference attacks against retrieval-augmented generation. arXiv preprint arXiv:2405.20446 .

Microsoft Research, 2025. LazyGraphRAG: Setting a New Standard for Quality and Cost. Technical Report. Microsoft Research. Microsoft Research Blog and Technical Report.

Peng, B., Galley, M., He, P., Cheng, H., Xie, Y., Hu, Y., Huang, Q., Liden, L., Yu, Z., Chen, W., Gao, J., 2024. Graph retrieval-augmented generation: A survey. ACM Transactions on Information Systems ArXiv:2408.08921.

PolyG Authors, 2025. PolyG: Adaptive graph traversal for diverse GraphRAG questions. arXiv preprint arXiv:2504.02112 .

PricewaterhouseCoopers, 2024. Graph LLMs in Banking and Insurance. Technical Report. PwC.

Privacy-Aware RAG Authors, 2025. Privacy-aware retrieval-augmented generation. arXiv preprint arXiv:2503.15548 .

Privacy Risks Authors, 2025. Privacy risks in GraphRAG. arXiv preprint arXiv:2508.17222 .

RAG Security Authors, 2025. A comprehensive threat model for RAG security. arXiv preprint arXiv:2509.20324 .

Role of RAG Authors, 2025. Role of retrieval-augmented generation in financial document processing. International Journal of Management and Technology (IJMT) 12.

SAG Authors, 2025. SAG: Provably secure retrieval-augmented generation. arXiv preprint arXiv:2508.01084 .

Sarmah, B., Hall, B., Rao, R., Patel, S., Pasquali, S., Mehta, D., 2024. HybridRAG: Integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. arXiv preprint arXiv:2408.04948 BlackRock and NVIDIA.

Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., Manning, C.D., 2024. RAPTOR: Recursive abstractive processing for tree-organized retrieval, in: Proceedings of the International Conference on Learning Representations (ICLR). ArXiv:2401.18059.

TERAG Authors, 2025. TERAG: Token-efficient graph-based retrieval-augmented generation. arXiv preprint arXiv:2509.18667 .

Traag, V.A., Waltman, L., van Eck, N.J., 2019. From Louvain to Leiden: Guaranteeing well-connected communities. Scientific Reports 9, 5233.

Unbiased Evaluation Authors, 2025. An unbiased evaluation framework for GraphRAG. arXiv preprint arXiv:2506.06331 .

Vector Institute, 2024. KG-RAG for SEC 10-Q financial document analysis. GitHub Repository. Vector Institute for Artificial Intelligence.

When-to-Use Authors, 2026. When to use graphs in retrieval-augmented generation, in: Proceedings of the International Conference on Learning Representations (ICLR). ArXiv:2506.05690.

Zhao, Y., et al., 2025. E2GraphRAG: Streamlining graph-based retrieval-augmented generation for high efficiency and effectiveness. arXiv preprint arXiv:2505.24226 .

Zhu, Y., et al., 2025. KG2RAG: Knowledge graph-guided retrieval-augmented generation, in: Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL).

Algorithm 2 Multi-Strategy Intent-Aware Retrieval

Require: Query q, security context σ, knowledge graph G, communities Γ, weight table $\mathbf{W}$, token budget B

Ensure: Ranked context $\mathcal{R}$, intent ι

```
// Authentication and Authorization
Verify(σ)                                   ▷ JWT validation, RBAC check
G_σ ← TenantFilter(G, σ.tenant_id)          ▷ Restrict to tenant subgraph
// Intent Classification
ι ← ClassifyIntent(q)                       ▷ Pattern matching for 6 intent types
E_q ← ExtractEntities(q, A)                 ▷ Longest-match-first
(w_g, w_v, w_c) ← W[ι]                      ▷ Intent-specific weights
// Parallel Retrieval
R_g ← GraphRetriever(G_σ, E_q, ι) in parallel
R_v ← VectorRetriever(q, G_σ, ι) in parallel
R_c ← CommunityRetriever(q, E_q, Γ) in parallel
// Fusion and Ranking
for each result r ∈ R_g ∪ R_v ∪ R_c do
    score(r) ← w_g · ŝ_g(r) + w_v · ŝ_v(r) + w_c · ŝ_c(r)   ▷ ŝ: min-max normalized scores
end for
R ← Deduplicate(R_g ∪ R_v ∪ R_c)            ▷ Keep highest score per chunk
R ← SortDescending(R, score)
R ← TruncateByBudget(R, B, min_results = 3)
R ← PermissionFilter(R, σ)                  ▷ Post-retrieval security check
return R, ι
```